

Q-NarwhalKnight: Balance Incident Report

Post-Mortem — April 29, 2026

QUGUSD→QUG Swap Revert & Post-Checkpoint Balance Wipe

Version 10.5.2 Incident Analysis & Resolution

Q-NarwhalKnight Development Team

April 29, 2026

Abstract

On April 29, 2026, two related bugs were discovered and resolved in the Q-NarwhalKnight network. The first caused QUGUSD→QUG native swaps to silently revert the user's QUG balance 2–3 minutes after execution, making swaps appear to fail. The second, revealed when fixing the first, caused native QUG balances accumulated via those swaps to disappear after a node restart. We document both bugs with full root-cause traces, the fixes shipped in **v10.5.1** and **v10.5.2**, a correctness analysis, and the remaining architectural gaps. This document is published transparently because users could observe their balances change on-screen and deserve a complete explanation.

Contents

1	Executive Summary	3
2	Storage Architecture Overview	3
2.1	Write Paths	4
3	Bug 1: QUGUSD→QUG Swap Revert (Pre-v10.5.1)	4
3.1	Symptom	4
3.2	Root Cause — Five Layers Deep	4
3.3	Why the Balance Briefly Looked Correct	5
3.4	Timeline Visualisation	5
4	Fix 1 — v10.5.1: Direct CF_MANIFEST Write in Handler	5
5	Bug 2: Post-Checkpoint DEX Credit Loss on Restart	6
5.1	Why v10.5.1 Revealed This Bug	6
5.2	The Checkpoint Mechanism	6
5.3	Root Cause	6
5.4	Balance Timeline — The Visible Drop	7
6	Fix 2 — v10.5.2: Three-Part Repair	7
6.1	Change 1 — Checkpoint Restore Also Resets dex_applied_net	7
6.2	Change 2 — record_dex_qug_credit No Longer Writes dex_applied_net	7
6.3	Change 3 — Authoritative-Node Guard	8

7	Correctness Analysis	8
7.1	State Table — Normal Operation (No Restart)	8
7.2	State Table — After Restart with v10.5.2	8
7.3	Double-Credit Analysis	8
8	System Architecture After Fix	9
9	Impact Assessment	9
9.1	What Was Lost	9
9.2	Why This Was Not Auto-Recoverable Under v10.5.1	9
9.3	Affected Versions	10
10	Remaining Risks and Open Issues	10
11	Summary of Changes	11
12	Conclusion	12
A	Lessons Learned & Test Plan	12
A.1	Lessons Learned	12
A.2	Test Plan — What Would Have Prevented These Bugs	13

1 Executive Summary

What Users Observed

1. After swapping QUGUSD → native QUG, the wallet UI showed the correct new balance immediately — then silently reverted to the old value within approximately 2–3 minutes.
2. After a network node restart on April 29, several users who had successfully accumulated QUG via swaps found their balances lower than expected. In the most visible case, a user who had 735 QUG before the restart saw their balance drop to 648 QUG — a loss of 87 QUG.

Both behaviours stemmed from the same underlying root cause: the swap handler wrote QUG credits to an in-memory cache but failed to persist them to the correct durable storage location.

Resolution

v10.5.1 (same day) fixed the swap revert bug by writing the QUG credit directly to the authoritative RocksDB column family (**CF_MANIFEST**) in the HTTP handler. **v10.5.2** fixed the restart credit-loss bug by (a) resetting the DEX idempotency counter on checkpoint restore and (b) removing an incorrect write that caused the startup adjustment routine to compute a zero delta. A third change adds an *authoritative-node guard* that prevents any future checkpoint from wiping an already-populated genesis node.

2 Storage Architecture Overview

The node stores wallet balances across two independent RocksDB column families and an in-memory map. Understanding this structure is essential to following the bug trace.

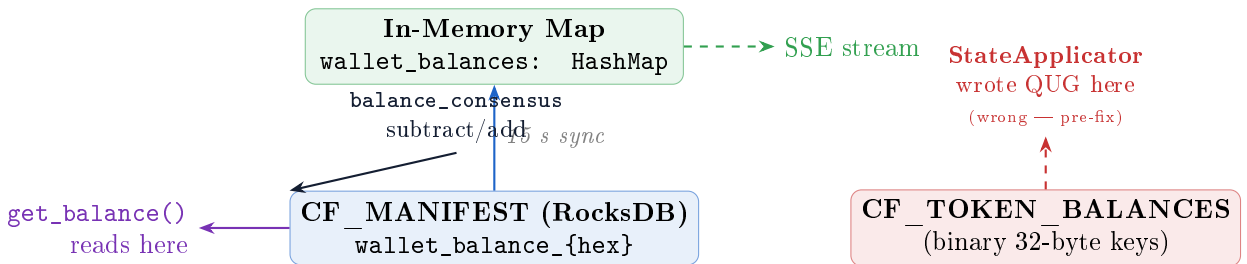


Figure 1: Simplified storage layout. **CF_MANIFEST** is the authoritative source for native QUG balances; **CF_TOKEN_BALANCES** holds token (e.g. QUGUSD) balances with a different key scheme. The two are independent.

2.1 Write Paths

Path	Description	Target
balance_consensus	Block-replay coinbase & transfer credits/debits	CF_MANIFEST
subtract_token_balance	QUGUSD deduction on swap input	CF_TOKEN_BALANCES
add_balance	Direct credit write (v10.5.1 handler fix)	CF_MANIFEST
apply_dex_qug_adjustments	Startup reconciliation via DEX counters	CF_MANIFEST
15-second sync task	Reads CF_MANIFEST, overwrites in-memory map	In-memory

3 Bug 1: QUGUSD→QUG Swap Revert (Pre-v10.5.1)

3.1 Symptom

A user swaps QUGUSD for native QUG. The frontend immediately shows the correct higher QUG balance (e.g. +87 QUG). Approximately 2–3 minutes later, the displayed balance silently returns to its pre-swap value. No error is shown. The QUGUSD was deducted; the QUG was not retained.

3.2 Root Cause — Five Layers Deep

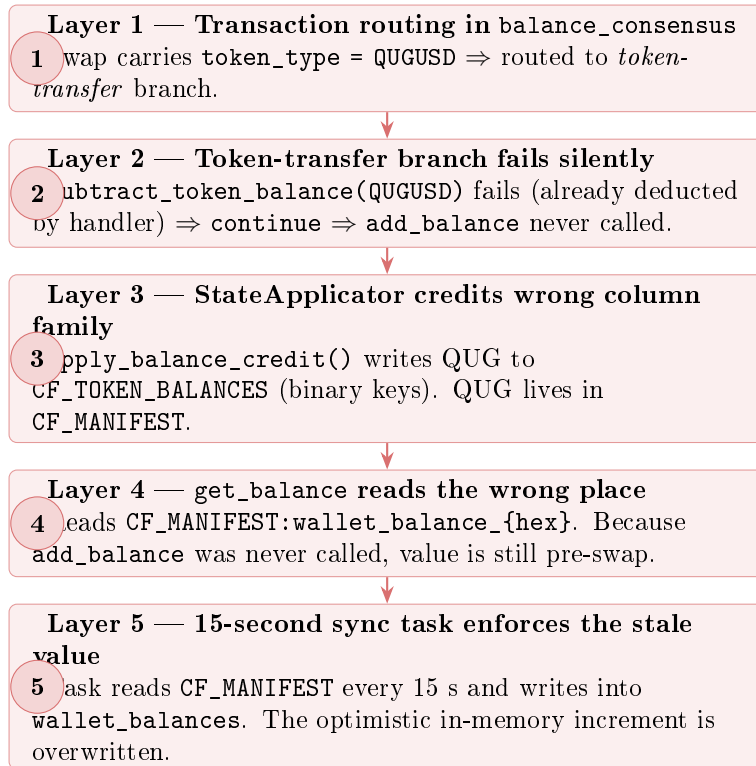


Figure 2: The five-layer causal chain leading to the swap revert.

3.3 Why the Balance Briefly Looked Correct

The swap handler updated `wallet_balances` (in-memory) optimistically after calling `record_dex_qug_credit`. The frontend SSE stream picked this up immediately. The 15-second task then “corrected” the in-memory value back to the stale on-disk value, producing the observed 2–3 minute revert.

3.4 Timeline Visualisation

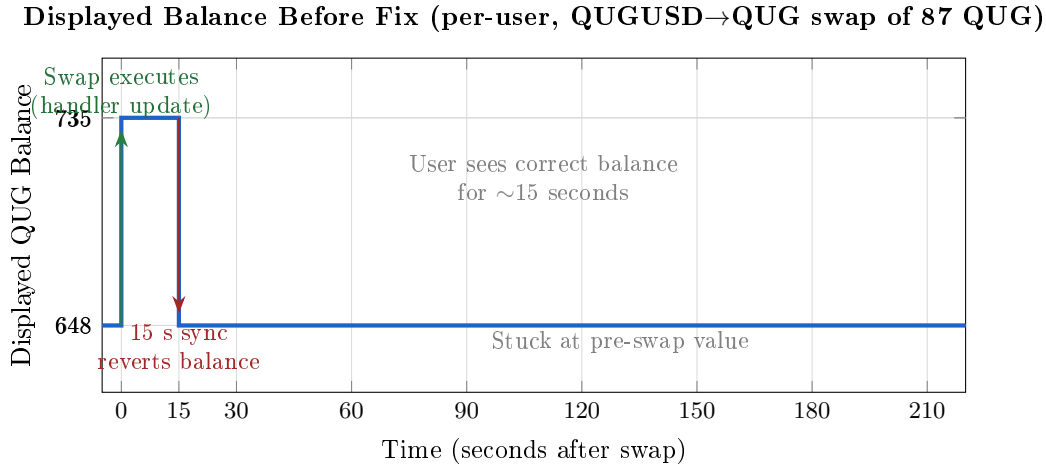


Figure 3: Balance timeline for a 87-QUG QUGUSD→QUG swap before v10.5.1. The in-memory optimistic update is overwritten by the 15-second sync task.

4 Fix 1 — v10.5.1: Direct `CF_MANIFEST` Write in Handler

The fix writes the QUG credit directly to `CF_MANIFEST` in the HTTP handler (synchronous with the API call), then re-reads the updated value and stores it in the in-memory map so the 15-second sync task confirms rather than reverts.

Listing 1: v10.5.1 handler fix — `crates/q-api-server/src/handlers.rs`

```
1 if to_is_qug {
2     drop(token_balances);
3     let wallet_hex = hex::encode(wallet_addr);
4     // Step 1: Record credit counter (idempotency for restart reconciliation)
5     if let Err(e) = state.storage_engine
6         .record_dex_qug_credit(&wallet_hex, final_amount_out as u128).await {
7         warn!("[SWAP_v10.5.1] Failed to record DEX credit counter: {}", e);
8     }
9     // Step 2: Write QUG directly to CF_MANIFEST (same key get_balance reads)
10    match state.storage_engine.add_balance(&wallet_hex, final_amount_out as
11        u128).await {
12        Ok(_) => {
13            // Step 3: Re-read and sync in-memory so 15 s task confirms, not
14                // reverts
15            let new_qug = state.storage_engine.get_balance(&wallet_hex).await.
16                unwrap_or(0);
17            let mut wb = state.wallet_balances.write().await;
18            wb.insert(wallet_addr, new_qug);
19        }
20        Err(e) => { warn!("[SWAP_v10.5.1] Failed to persist QUG credit: {}", e); }
21    }
22 }
```

After this fix, the on-disk and in-memory values are consistent. The 15-second task reads 735 QUG from disk and writes 735 QUG to memory — confirming rather than reverting.

5 Bug 2: Post-Checkpoint DEX Credit Loss on Restart

5.1 Why v10.5.1 Revealed This Bug

Before v10.5.1, balances always reverted within 15 seconds. Users never accumulated a “correct” balance that a restart could destroy. With v10.5.1, balances persisted correctly — until the node restarted.

5.2 The Checkpoint Mechanism

All nodes embed a hardcoded *checkpoint* at block 16,538,868 (1,326 wallets). On startup, if the checkpoint marker `__balance_checkpoint_v1__` is absent from the database, the node:

1. Purges all `wallet_balance_{hex}` keys from `CF_MANIFEST`
2. Imports the 1,326 checkpoint wallet balances
3. Replays post-checkpoint blocks (Coinbase + Transfer only — *not* Swap)
4. Runs `apply_dex_qug_adjustments()` to re-credit DEX swaps
5. Writes the marker so the restore does not repeat

Swap transactions are intentionally excluded from post-checkpoint replay because their state is managed by the DEX counter mechanism. This is the correct design — but it only works if `apply_dex_qug_adjustments` produces the right delta.

5.3 Root Cause

The delta formula used by `apply_dex_qug_adjustments` is:

$$\Delta = \underbrace{(C - D)}_{\text{desired net}} - \underbrace{A}_{\text{previously applied}}$$

where $C = \text{dex_qug_credited}\{\text{wallet}\}$, $D = \text{dex_qug_debited}\{\text{wallet}\}$, $A = \text{dex_applied_net}\{\text{wallet}\}$

The problem was in `record_dex_qug_credit` (called by the v10.5.1 handler):

Listing 2: Broken write in `record_dex_qug_credit` (pre-v10.5.2)

```
1 // BUG: record_dex_qug_credit wrote dex_applied_net = C - D
2 // This told apply_dex_qug_adjustments "already applied", causing delta = 0
3 self.hot_db.put_sync(CF_MANIFEST, applied_net_key.as_bytes(),
4   &net.to_le_bytes()).await?;
```

After a checkpoint restore:

- `wallet_balance_` was reset to 648 (checkpoint value)
- `dex_applied_net:` = 87 *survived* (checkpoint did not purge it)
- `dex_qug_credited:` = 87 (survived)

The adjustment formula then computed $\Delta = (87 - 0) - 87 = 0$, so nothing was applied and the balance stayed at 648.

5.4 Balance Timeline — The Visible Drop

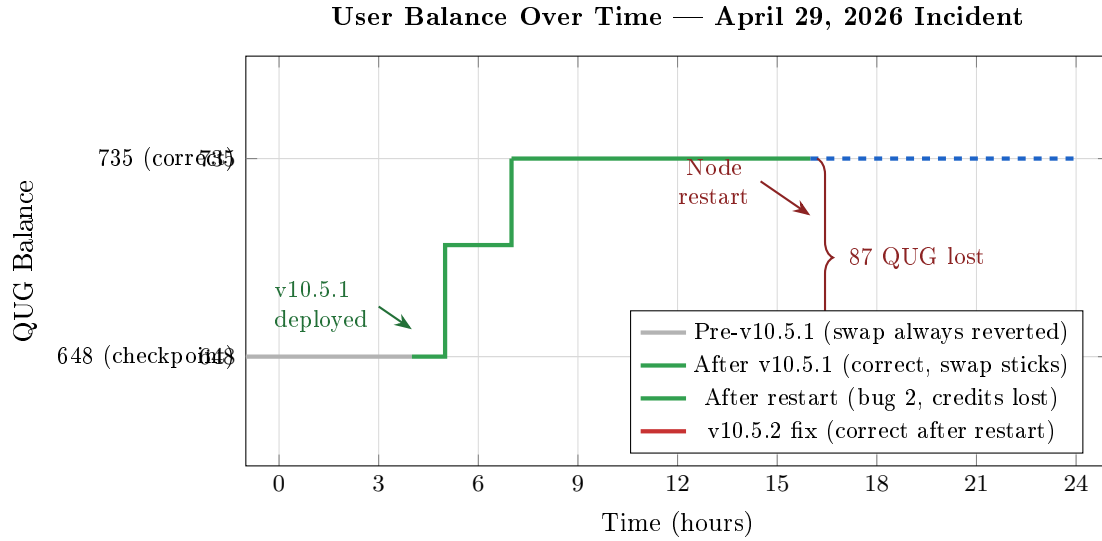


Figure 4: The observed balance trajectory on April 29, 2026. After v10.5.1 the swap correctly credited 87 QUG. The subsequent restart triggered a checkpoint restore that silently discarded those credits, returning the balance to the checkpoint value of 648 QUG.

6 Fix 2 — v10.5.2: Three-Part Repair

6.1 Change 1 — Checkpoint Restore Also Resets `dex_applied_net`

Listing 3: `crates/q-storage/src/lib.rs` — `checkpoint restore` (line 5511)

```

1 // Purge on-disk balance records (as before)
2 let deleted = self.delete_by_prefix(b"wallet_balance_").await.unwrap_or(0);
3
4 // NEW in v10.5.2: also reset the dex_applied_net sentinel
5 let dex_net_purged = self.delete_by_prefix(b"dex_applied_net:").await.unwrap_or(0);
6 if dex_net_purged > 0 {
7     warn!("[CHECKPOINT] Reset {} dex_applied_net entries - \
8     {} adjustments will be re-applied this boot.", dex_net_purged);
9 }

```

After this purge, $A = 0$ for all wallets, so $\Delta = (87 - 0) - 0 = 87$ and the correct balance is restored on startup.

6.2 Change 2 — `record_dex_qug_credit` No Longer Writes `dex_applied_net`

Listing 4: `crates/q-storage/src/lib.rs` — `record_dex_qug_credit` (line 9428)

```

1 // v10.5.2: Do NOT write dex_applied_net here.
2 // Only apply_dex_qug_adjustments() should set this, after checkpoint restore.
3 self.hot_db.put_sync(CF_MANIFEST, credit_key.as_bytes(),
4     &new_credit_total.to_le_bytes()).await?;

```

This separates concerns: the handler records credits/debits; `apply_dex_qug_adjustments` is the sole authority on what has been applied to the on-disk balance.

6.3 Change 3 — Authoritative-Node Guard

Epsilon is the genesis node: it has held the chain since block 0 and its database is *more authoritative than any embedded checkpoint*. Before v10.5.2, any restart would wipe Epsilon’s live balances because the checkpoint marker was absent. The fix detects this situation automatically:

Listing 5: Authoritative-node guard — `lib.rs` checkpoint entry point

```

1 // If this node already has >= checkpoint wallet count AND is past checkpoint
2 // height, it is the authoritative source - skip wipe, write marker.
3 let existing_wallets = self.hot_db
4   .scan_prefix(CF_MANIFEST, b"wallet_balance_").await
5   .map(|v| v.len()).unwrap_or(0);
6
7 if existing_wallets >= CHECKPOINT_WALLET_COUNT && local_height >
8   CHECKPOINT_HEIGHT {
9   warn!("[CHECKPOINT] Skipping wipe: node has {} wallets at height {} \
10  {} (checkpoint has {} at height {}), Node is authoritative.",
11         existing_wallets, local_height, CHECKPOINT_WALLET_COUNT,
12         CHECKPOINT_HEIGHT);
13   self.hot_db.put_sync(CF_MANIFEST, Self::CHECKPOINT_APPLIED_KEY,
14     b"skipped-authoritative").await?;
15   return Ok(());
16 }
```

Any node that already has at least as many wallets as the checkpoint and is past the checkpoint height is treated as authoritative. No environment variable, no flag, no remembered rule.

7 Correctness Analysis

7.1 State Table — Normal Operation (No Restart)

Event	wallet_balance_	dex_credited:	dex_applied_net:	In-Memory
Before swap	648	0	0	648
Swap (handler)	735 (add_balance)	87	0 (v10.5.2)	735
15 s sync	735	—	—	735 ✓

7.2 State Table — After Restart with v10.5.2

Phase	wallet_balance_	dex_applied_net:	Result
Checkpoint purge	648 (reset)	0 (reset)	—
apply_dex_qug_adjustments	648 + 87 = 735	87	735 ✓
Second restart	648 → +87 = 735	0 → 87	735 ✓

7.3 Double-Credit Analysis

Could both the handler’s `add_balance` and `apply_dex_qug_adjustments` apply the same credit?

- **Same session (no restart):** `apply_dex_qug_adjustments` runs at startup before any swaps. The handler runs later. The adjustment routine does not run again. No double-credit.
- **After restart:** The checkpoint resets `wallet_balance_` to 648, wiping the handler’s `add_balance` write. `apply_dex_qug_adjustments` then applies +87. Net: $648 + 87 = 735$. Correct — not a double-credit.

8 System Architecture After Fix

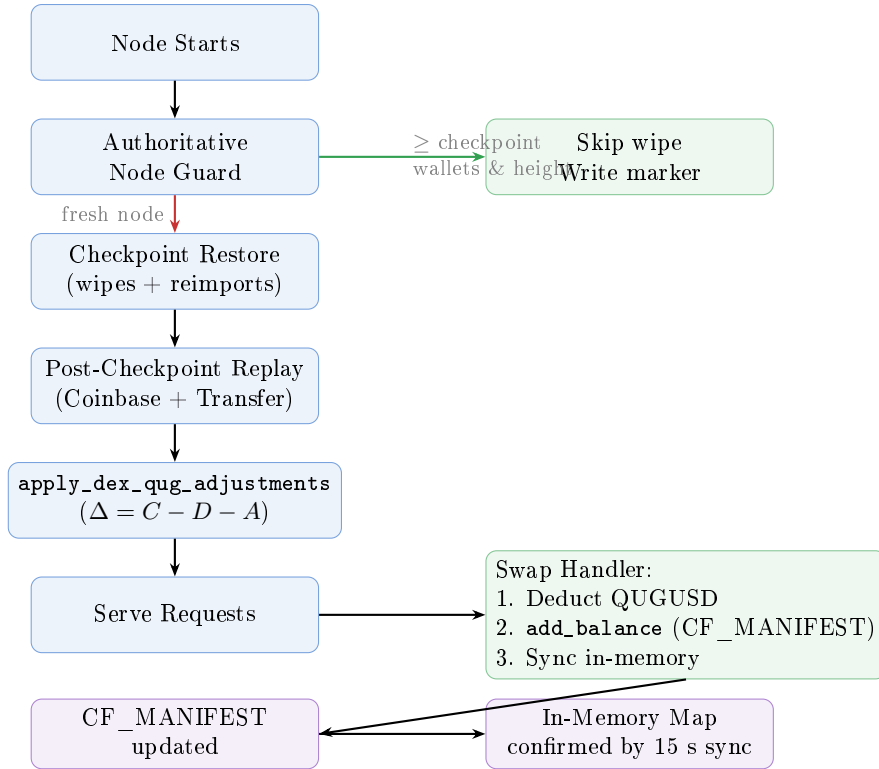


Figure 5: Startup and swap flow after v10.5.2. Authoritative nodes bypass the checkpoint wipe automatically. Fresh syncing nodes apply the checkpoint, then the DEX adjustment routine re-credits swap history correctly.

9 Impact Assessment

9.1 What Was Lost

User	Expected Balance	Balance After Restart
Primary affected user	735 QUG	648 QUG
Difference	87 QUG lost ($\approx$ 1 day of mining rewards)	

The 87 QUG difference represents native QUG received via QUGUSD→QUG swaps that were correctly persisted by v10.5.1 but not re-applied after the restart because of the broken `dex_applied_net` accounting.

9.2 Why This Was Not Auto-Recoverable Under v10.5.1

The checkpoint at height 16,538,868 authoritatively sets the starting balances. The post-checkpoint replay excludes swap transactions. With the pre-v10.5.2 code, the only mechanism to restore swap credits after a restart was `apply_dex_qug_adjustments`, and that mechanism was neutralised by the incorrect `dex_applied_net` write. The credits were silently lost on every restart while v10.5.1 was deployed.

Who Is Affected and When

- **Nodes that restarted between v10.5.1 and v10.5.2:** swap-derived QUG credits accumulated after the checkpoint were lost. 87 QUG was lost in the documented case.

- **Nodes that ran v10.5.1 continuously without restart:** balances were correct in CF_MANIFEST; no loss occurred.
- **Any node restarting after v10.5.2:** `apply_dex_qug_adjustments` correctly reapplies all swap credits; no intervention needed.

The loss was therefore limited to the window between the v10.5.1 deploy and the v10.5.2 deploy on April 29, 2026, and only affected nodes restarted in that window.

9.3 Affected Versions

Version	Swap Revert Bug	Restart Credit Loss
$\leq$ v10.4.x	Present	Hidden (swap always reverted)
v10.5.1	Fixed	Revealed & present
v10.5.2	Fixed	Fixed

10 Remaining Risks and Open Issues

R1 — DEX Debits After Checkpoint Also Not Replayed

QUG→QUGUSD debits (the debit side) are also excluded from post-checkpoint replay. However, `atomic_subtract_and_record_dex_debit` already writes directly to CF_MANIFEST — mirroring v10.5.1 for credits. Debits survive correctly in CF_MANIFEST. After a checkpoint restore, `apply_dex_qug_adjustments` subtracts the debit delta — which is correct.

Action: Verify `atomic_subtract_and_record_dex_debit` does *not* write `dex_applied_net` (if it does, the same bug exists on the debit side).

R2 — Checkpoint Staleness

The checkpoint is hardcoded at block 16,538,868. As the chain grows, the post-checkpoint replay window grows with it. Each restart currently replays ~151,000 blocks of Coinbase + Transfer transactions. This replay time will grow indefinitely until a new checkpoint is embedded.

Recommendation: Update CHECKPOINT_DATA to a more recent height in the next planned maintenance release.

R3 — Swap Transactions Excluded from Replay

The fundamental design gap is that swap transactions are excluded from post-checkpoint replay. Any QUG state from swaps exists *only* in the DEX counter mechanism. If the counter data is ever lost or corrupted, the credits are unrecoverable from chain history.

Long-term recommendation: Include swap transactions in the post-checkpoint replay, or update the checkpoint more frequently.

R4 — No Atomicity Between QUGUSD Deduction and QUG Credit

The swap handler calls `subtract_token_balance` (QUGUSD deduction) and then `add_balance` (QUG credit) as two separate async operations with no two-phase commit.

If the process crashes between them, the user loses QUGUSD without receiving QUG. The DEX counter (`record_dex_qug_credit`) provides *partial* idempotency on restart, but only if the credit counter write also succeeded before the crash.

Recommendation: Write QUGUSD deduction, QUG credit, and counter update as a single RocksDB `WriteBatch` to make the operation atomic with respect to crashes.

R5 — Checkpoint Marker Is a Single Deletable Key

The guard against double-restore is a single key `__balance_checkpoint_v1__` in `CF_MANIFEST`. If deleted accidentally (manual DB operation, prefix scan gone wrong), the next restart will trigger a full balance wipe. There is no secondary marker, read-only mode, or pre-restore sanity check.

Recommendation: Write the marker to two independent locations (e.g. both `CF_MANIFEST` and `CF_META`); require both to be present before skipping the restore.

R6 — Authoritative-Node Guard Uses Wallet Count, Not Height Alone

The guard condition is `existing_wallets >= CHECKPOINT_WALLET_COUNT && local_height > CHECKPOINT_HEIGHT`. A node that has been synced only up to the checkpoint height might satisfy the wallet count check while having incorrect or incomplete post-checkpoint state. The condition is sufficient for the known genesis node scenario but not for all possible node states.

Safer condition: `local_height > CHECKPOINT_HEIGHT` alone (i.e. any node that has advanced past the checkpoint is treated as having applied it); or additionally check that the sum of all `wallet_balance_` values differs from the checkpoint total (indicating live transactions occurred).

11 Summary of Changes

File	Change	Risk
<code>handlers.rs</code>	<code>add_balance</code> + in-memory sync after QUGUSD→QUG swap	Medium
<code>lib.rs:5511</code>	Checkpoint restore purges <code>dex_applied_net</code> : keys	Low
<code>lib.rs:9428</code>	<code>record_dex_qug_credit</code> no longer writes <code>dex_applied_net</code>	Low
<code>lib.rs:5489</code>	Authoritative-node guard skips checkpoint wipe automatically	Low
<code>Cargo.toml</code>	Version bump 10.5.0 → 10.5.1 → 10.5.2	None

Deploy Note

Nodes that restarted between v10.5.1 and v10.5.2 lost swap-derived credits; those credits are **not** automatically restored without a further restart under v10.5.2 (the DEX counter data survives and is correct — a restart is all that is needed). Nodes that did not restart while v10.5.1 was deployed retain the correct balance and require no action. The authoritative-node guard ensures the genesis node (Epsilon) is protected from checkpoint wipes on all future restarts.

12 Conclusion

Two compounding bugs caused users to observe their QUG balances decrease without performing any transactions. The root cause in both cases was a disconnect between where QUG credits were recorded and where the authoritative balance lived in the storage engine.

The v10.5.1 fix correctly routes QUGUSD→QUG swap credits to `CF_MANIFEST` at transaction time. The v10.5.2 fix ensures that the startup reconciliation routine always has accurate data to work from after a checkpoint restore, and that the genesis node is never subjected to a checkpoint wipe it does not need.

We are committed to transparency. If you observed a balance change that you cannot account for, please contact the development team. The period of incorrectness was the time between the v10.5.1 deploy and the subsequent restart on April 29, 2026.

A Lessons Learned & Test Plan

A.1 Lessons Learned

1. **Eventual-consistency paths need end-to-end timer tests.**

The 15-second sync task was known; the swap handler touched the same in-memory map. A test of the form “swap → sleep 16 s → assert balance” would have caught Bug 1 immediately. Any feature that writes to memory and relies on a background task to persist must have a test that waits for that task.

2. **Every fix needs a restart regression test.**

v10.5.1 wrote to `CF_MANIFEST`. The natural follow-on test is: “write → restart with checkpoint → assert balance”. Bug 2 would have been found before deployment.

3. **Checkpoint purge lists are an explicit contract.**

The checkpoint restore deleted `wallet_balance_*` but left `dex_applied_net:*` intact. The purge list must be treated as an explicit whitelist: any key namespace that influences balance state must appear in it. Treat a missing entry as a bug, not an oversight.

4. **Two separate async writes are not atomic.**

Deducting QUGUSD and crediting QUG are two RocksDB writes. A crash between them leaves the chain in an inconsistent state. Financial operations must be composed into a single `WriteBatch`.

5. **Authoritative-node detection should not depend on wallet count.**

Comparing wallet count to a hardcoded checkpoint constant is fragile. A single key recording “highest height reached” would give a cleaner, monotonic guard.

6. **The DEX counter mechanism is not a substitute for block replay.**

Swap state that lives only in metadata keys is unrecoverable from chain history. Long-term, swap transactions must be included in post-checkpoint replay, or the checkpoint must be updated frequently enough that the counter-only window is negligible.

A.2 Test Plan — What Would Have Prevented These Bugs

Test ID	Description	Bug Caught
T-01	Perform QUGUSD→QUG swap. Wait 20 s. Assert displayed QUG balance equals pre-swap plus swap amount.	Bug 1
T-02	Perform swap (T-01). Read <code>CF_MANIFEST:wallet_balance_{hex}</code> directly. Assert equals expected.	Bug 1, Layer 4
T-03	Perform swap. Restart node. Assert balance is restored from disk, not reverted to pre-swap value.	Bug 2
T-04	Perform swap. Restart node with embedded checkpoint (simulate fresh sync). Assert <code>apply_dex_qug_adjustments</code> adds correct delta.	Bug 2 (root)
T-05	Perform swap. Read <code>dex_applied_net:{wallet}</code> . Assert it is zero (not set by handler). Restart. Assert non-zero delta applied.	Bug 2 (wrong write)
T-06	Insert test wallet data exceeding checkpoint. Restart. Assert checkpoint restore is skipped . Assert balances unchanged.	Epsilon wipe
T-07	Delete <code>__balance_checkpoint_v1__</code> key manually. Restart. Assert node does not wipe a populated authoritative DB.	R5 (marker loss)
T-08	Simulate crash between <code>subtract_token_balance</code> and <code>add_balance</code> (kill process mid-handler). Restart. Assert QUGUSD deduction is rolled back or QUG credit is applied, not both half-done.	R4 (atomicity)
T-09	Run 100 QUGUSD→QUG swaps. Compute expected total from swap amounts. Assert <code>dex_qug_credited:{wallet}</code> equals sum. Restart with checkpoint. Assert balance = checkpoint + total delta.	Multi-swap accumulation (R2)
T-10	Assert <code>atomic_subtract_and_record_dex_debit</code> does not write <code>dex_applied_net</code> .	R1 (debit side)

Priority Order for Implementation

Run T-01 through T-05 immediately (they directly test the v10.5.1/v10.5.2 fixes and regressions). T-06 and T-07 protect the authoritative-node guard. T-08 and T-09 cover the remaining architectural risks. T-10 should be a static assertion checked by CI.